

# SQL for Business Analysts — Complete Interview Guide

DDL · DML · DCL · TCL · DQL · Keys · Constraints · Joins · Windows · Normalization · ACID · Q&A. Capital-markets / corporate-actions flavored.

1. SQL command categories · 2. Data types · 3. DDL (CREATE/ALTER/DROP/TRUNCATE) · 4. Keys (all types) · 5. Constraints · 6. DML (INSERT/UPDATE/DELETE/MERGE) · 7. TCL · 8. DCL · 9. Sample tables · 10. SELECT basics · 11. Operators & NULL · 12.
- Aggregation · 13. Joins · 14. Set operations · 15. Subqueries · 16. Window functions · 17. CTEs · 18. Built-in functions · 19. Views/Index/Proc/Trigger · 20. Normalization · 21. ACID & transactions · 22. Conceptual Q&A · 23. Practice prompts

## 1. SQL Command Categories

Every SQL statement falls into one of five families. Being able to name them and give an example is a guaranteed BA question.

| Category | Full form                    | Purpose                            | Commands                              | Auto-commit?          |
|----------|------------------------------|------------------------------------|---------------------------------------|-----------------------|
| DDL      | Data Definition Language     | Define / change structure (schema) | CREATE, ALTER, DROP, TRUNCATE, RENAME | Yes (implicit commit) |
| DML      | Data Manipulation Language   | Change the data in tables          | INSERT, UPDATE, DELETE, MERGE         | No                    |
| DQL      | Data Query Language          | Read data                          | SELECT                                | No                    |
| DCL      | Data Control Language        | Permissions / access               | GRANT, REVOKE                         | Yes                   |
| TCL      | Transaction Control Language | Manage transactions                | COMMIT, ROLLBACK, SAVEPOINT           | —                     |

**Memory hook:** DDL = structure, DML = data, DQL = read, DCL = who-can-do-what, TCL = save-or-undo. DDL is auto-committed — you cannot ROLLBACK a DROP/TRUNCATE.

## 2. Common Data Types

| Group            | Types                                                    | Notes                                                    |
|------------------|----------------------------------------------------------|----------------------------------------------------------|
| Numeric          | INT, BIGINT, SMALLINT, DECIMAL(p,s)/NUMERIC, FLOAT, REAL | Use DECIMAL for money (exact); never FLOAT for currency. |
| Character        | CHAR(n), VARCHAR(n), TEXT                                | CHAR = fixed length (padded), VARCHAR = variable.        |
| Date/Time        | DATE, TIME, DATETIME/TIMESTAMP                           | TIMESTAMP often stores timezone/UTC.                     |
| Boolean / Binary | BOOLEAN, BLOB, BYTEA                                     | Boolean stored as 0/1 in some engines.                   |

**Trap:** CHAR(10) for "AB" stores "AB\_" (padded to 10). Use VARCHAR unless the length is truly fixed (e.g., ISIN = 12 chars, currency code = 3 chars).

## 3. DDL — Data Definition Language

### 3.1 CREATE TABLE (with constraints inline)

```
CREATE TABLE clients (  
  client_id      INT           PRIMARY KEY,  
  client_name    VARCHAR(100) NOT NULL,  
  country        CHAR(2)      DEFAULT 'US',  
  email          VARCHAR(150) UNIQUE,  
  aum            DECIMAL(18,2) CHECK (aum >= 0),  
  created_at     TIMESTAMP    DEFAULT CURRENT_TIMESTAMP  
);
```

### 3.2 CREATE with a FOREIGN KEY (referential integrity)

```
CREATE TABLE trades (  
  trade_id INT PRIMARY KEY,  
  client_id INT NOT NULL,  
  symbol VARCHAR(10) NOT NULL,  
  qty INT CHECK (qty > 0),  
  price DECIMAL(12,2),  
  trade_date DATE DEFAULT CURRENT_DATE,  
  CONSTRAINT fk_client  
    FOREIGN KEY (client_id) REFERENCES clients(client_id)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

**Referential actions:** ON DELETE CASCADE deletes child rows when the parent is deleted; SET NULL nulls the FK; RESTRICT/NO ACTION blocks the delete if children exist.

### 3.3 ALTER TABLE — the most-asked DDL

```
ALTER TABLE clients ADD COLUMN risk_rating VARCHAR(10); -- add column  
ALTER TABLE clients DROP COLUMN risk_rating; -- remove column  
ALTER TABLE clients ALTER COLUMN country SET DEFAULT 'GB'; -- change default  
ALTER TABLE clients RENAME COLUMN aum TO assets_under_mgmt; -- rename column  
ALTER TABLE trades ADD CONSTRAINT chk_price CHECK (price > 0); -- add constraint  
ALTER TABLE trades DROP CONSTRAINT fk_client; -- drop constraint
```

### 3.4 DROP / TRUNCATE / RENAME

```
TRUNCATE TABLE trades; -- delete ALL rows, fast, keep structure, no WHERE  
DROP TABLE trades; -- delete table entirely (structure + data)  
ALTER TABLE trades RENAME TO trade_book;
```

**Classic 3-way question — DELETE vs TRUNCATE vs DROP:**

|                 | DELETE               | TRUNCATE          | DROP        |
|-----------------|----------------------|-------------------|-------------|
| Type            | DML                  | DDL               | DDL         |
| Removes         | Rows (WHERE allowed) | All rows          | Whole table |
| WHERE clause    | Yes                  | No                | No          |
| Rollback        | Yes                  | No (auto-commit)  | No          |
| Resets identity | No                   | Yes               | N/A         |
| Speed / logging | Slow, fully logged   | Fast, minimal log | Fast        |
| Fires triggers  | Yes                  | No                | No          |

## 4. Keys — Every Type a BA Should Name

| Key                  | Definition                                                               | Example (trades/clients)                      |
|----------------------|--------------------------------------------------------------------------|-----------------------------------------------|
| <b>Super key</b>     | Any set of columns that uniquely identifies a row (may have extra cols)  | {client_id}, {client_id, email}               |
| <b>Candidate key</b> | Minimal super key — no redundant column                                  | client_id, email (both can identify a client) |
| <b>Primary key</b>   | The chosen candidate key; UNIQUE + NOT NULL; one per table               | client_id                                     |
| <b>Alternate key</b> | Candidate keys not chosen as PK                                          | email                                         |
| <b>Composite key</b> | PK made of 2+ columns together                                           | (client_id, symbol, trade_date)               |
| <b>Foreign key</b>   | Column referencing a PK in another table; enforces referential integrity | trades.client_id → clients.client_id          |
| <b>Unique key</b>    | Enforces uniqueness; allows one NULL; many per table                     | email                                         |
| <b>Surrogate key</b> | System-generated meaningless id (auto-increment/GUID)                    | trade_id (identity)                           |
| <b>Natural key</b>   | Business-meaningful column used as identifier                            | ISIN, SWIFT/BIC, PAN                          |

**PK vs Unique (asked constantly):** Primary key = NOT NULL + unique, exactly one per table, defines the row's identity. Unique key = unique but allows a single NULL, and you can have many per table.

## 5. Constraints

| Constraint  | Enforces                                                             |
|-------------|----------------------------------------------------------------------|
| NOT NULL    | Column must have a value                                             |
| UNIQUE      | No duplicate values (one NULL allowed)                               |
| PRIMARY KEY | UNIQUE + NOT NULL; row identity                                      |
| FOREIGN KEY | Value must exist in referenced table (referential integrity)         |
| CHECK       | Value must satisfy a condition, e.g. <code>CHECK (qty &gt; 0)</code> |
| DEFAULT     | Value used when none supplied                                        |

## 6. DML — Data Manipulation Language

### 6.1 INSERT

```
-- Single row
INSERT INTO clients (client_id, client_name, country)
VALUES (5, 'Quantum Partners', 'SG');

-- Multiple rows
INSERT INTO clients (client_id, client_name, country) VALUES
  (6, 'Rhea Capital', 'US'),
  (7, 'Sterling AM', 'GB');

-- Insert from another table (INSERT ... SELECT)
INSERT INTO archived_trades
SELECT * FROM trades WHERE trade_date < '2023-01-01';
```

### 6.2 UPDATE

```
UPDATE clients
SET country = 'UK'
WHERE client_id = 1;

-- Update using a join / correlated subquery
UPDATE trades
SET price = price * 1.02
WHERE client_id IN (SELECT client_id FROM clients WHERE country = 'US');
```

**#1 real-world disaster:** `UPDATE trades SET price = 0;` with **no WHERE** updates **every row**. Same for DELETE. Always write the WHERE first, and SELECT it before you run the UPDATE.

### 6.3 DELETE

```
DELETE FROM trades WHERE trade_date < '2020-01-01';
DELETE FROM trades;           -- deletes all rows (but rollback-able, unlike TRUNCATE)
```

### 6.4 MERGE / UPSERT (insert-or-update)

```
-- ANSI MERGE (Oracle, SQL Server)
MERGE INTO clients c
USING staging s ON (c.client_id = s.client_id)
WHEN MATCHED THEN UPDATE SET c.client_name = s.client_name
WHEN NOT MATCHED THEN INSERT (client_id, client_name)
  VALUES (s.client_id, s.client_name);

-- Postgres / MySQL style
INSERT INTO clients (client_id, client_name) VALUES (5, 'Quantum')
ON CONFLICT (client_id) DO UPDATE SET client_name = EXCLUDED.client_name;
```

## 7. TCL — Transaction Control

```
BEGIN;                                -- start transaction
UPDATE accounts SET bal = bal - 100 WHERE id = 1;  -- debit
SAVEPOINT after_debit;                  -- checkpoint
UPDATE accounts SET bal = bal + 100 WHERE id = 2;  -- credit
ROLLBACK TO after_debit;                -- undo only the credit
COMMIT;                                 -- make changes permanent
-- ROLLBACK;                            -- undo the whole transaction
```

A transfer is the textbook transaction: both the debit and credit must succeed, or neither should. That "all-or-nothing" guarantee is **Atomicity** (see §21).

## 8. DCL — Data Control

```
GRANT SELECT, INSERT ON trades TO analyst_role;  -- give permission
REVOKE INSERT ON trades FROM analyst_role;       -- take it back
```

## 9. Sample Tables (used from here on)

employees

| emp_id | name   | dept           | mgr  | salary |
|--------|--------|----------------|------|--------|
| 1      | Asha   | Operations     | NULL | 95000  |
| 2      | Ben    | Operations     | 1    | 62000  |
| 3      | Carol  | Settlements    | 1    | 68000  |
| 4      | Dev    | Settlements    | 3    | 54000  |
| 5      | Ella   | Reconciliation | 1    | 71000  |
| 6      | Faisal | Reconciliation | 5    | 48000  |

clients

| client_id | client_name      | country |
|-----------|------------------|---------|
| 1         | Meridian Capital | UK      |
| 2         | Nova Funds       | US      |
| 3         | Orion AM         | India   |
| 4         | Pallas LLP       | US      |

trades

| trade_id | client_id | symbol | qty | price  | trade_date |
|----------|-----------|--------|-----|--------|------------|
| 101      | 1         | AAPL   | 100 | 180.00 | 2024-01-05 |
| 102      | 1         | MSFT   | 50  | 400.00 | 2024-01-06 |
| 103      | 2         | AAPL   | 200 | 182.00 | 2024-01-06 |
| 104      | 3         | TSLA   | 75  | 210.00 | 2024-01-07 |
| 105      | 2         | MSFT   | 30  | 405.00 | 2024-01-08 |

## 10. SELECT Basics (DQL)

```
SELECT name, dept, salary FROM employees WHERE salary > 60000;
SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3; -- top 3
SELECT DISTINCT dept FROM employees; -- unique depts
```

WHERE salary > 60000 → Asha, Ben, Carol, Ella

## 11. Operators & NULL Handling

```
WHERE dept IN ('Settlements','Reconciliation') -- set membership
WHERE salary BETWEEN 50000 AND 70000 -- inclusive range
WHERE name LIKE 'C%' -- starts with C _ = one char, % = any chars
WHERE manager_id IS NULL -- NULL test (never use = NULL)
```

**NULL logic (favourite trick):** NULL = NULL is **unknown**, not true. WHERE x = NULL returns nothing — use IS NULL. Also COUNT(col) ignores NULLs but COUNT(\*) counts every row.

## 12. Aggregation (GROUP BY / HAVING)

```
SELECT dept, COUNT(*) AS headcount, AVG(salary) AS avg_sal, MAX(salary) AS top
FROM employees
GROUP BY dept
HAVING AVG(salary) > 60000;
```

Output

| dept        | headcount | avg_sal | top   |
|-------------|-----------|---------|-------|
| Operations  | 2         | 78500   | 95000 |
| Settlements | 2         | 61000   | 68000 |

**WHERE vs HAVING:** WHERE filters rows *before* grouping (no aggregates). HAVING filters groups *after* aggregation. Aggregate functions: COUNT, SUM, AVG, MIN, MAX.

```
SELECT symbol, SUM(qty*price) AS notional FROM trades GROUP BY symbol;
-- AAPL 54400 (100*180 + 200*182), MSFT 32150, TSLA 15750
```

## 13. Joins

```
-- INNER: only matching rows (Pallas LLP has no trades -> excluded)
SELECT t.trade_id, c.client_name, t.symbol
FROM trades t INNER JOIN clients c ON t.client_id = c.client_id;

-- LEFT + IS NULL: the "anti-join" -> clients with NO trades
SELECT c.client_name
FROM clients c LEFT JOIN trades t ON c.client_id = t.client_id
WHERE t.trade_id IS NULL;           -- Pallas LLP

-- SELF JOIN: employee -> manager
SELECT e.name AS employee, m.name AS manager
FROM employees e LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

### Join types:

- **INNER** — only rows matching in both tables.
- **LEFT (OUTER)** — all left rows + matches (NULL where none).
- **RIGHT (OUTER)** — all right rows + matches.
- **FULL OUTER** — everything from both sides.
- **CROSS** — Cartesian product (every combination; rows =  $m \times n$ ).

**Why did INNER JOIN return fewer rows?** Non-matching keys and NULL keys are dropped. Conversely, a many-to-many join *inflates* rows ("row explosion") — check your join keys.

## 14. Set Operations

| Operator       | Returns                     | Dedups?      |
|----------------|-----------------------------|--------------|
| UNION          | All rows from both queries  | Yes (slower) |
| UNION ALL      | All rows incl. duplicates   | No (faster)  |
| INTERSECT      | Rows in <i>both</i> queries | Yes          |
| EXCEPT / MINUS | Rows in 1st but not 2nd     | Yes          |

```
SELECT client_id FROM trades WHERE symbol='AAPL'
INTERSECT
SELECT client_id FROM trades WHERE symbol='MSFT';  -- clients trading both
```

All set operators require the same number of columns with compatible data types. UNION vs JOIN: UNION stacks rows *vertically*; JOIN adds columns *horizontally*.

## 15. Subqueries

```
-- Scalar: above overall average
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);  -- Asha, Carol, Ella

-- Correlated: top earner per department (runs per outer row)
SELECT name, dept, salary FROM employees e
WHERE salary = (SELECT MAX(salary) FROM employees e2 WHERE e2.dept = e.dept);

-- EXISTS: clients who have traded
SELECT client_name FROM clients c
WHERE EXISTS (SELECT 1 FROM trades t WHERE t.client_id = c.client_id);
```

## 16. Window Functions (advanced — the senior differentiator)

Aggregate across a set of rows **without collapsing them** (unlike GROUP BY). Syntax: `func() OVER (PARTITION BY ... ORDER BY ...)`.

```
SELECT name, dept, salary,
       ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn,
       RANK() OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employees;
```

**ROW\_NUMBER vs RANK vs DENSE\_RANK:** ROW\_NUMBER = unique 1,2,3,4 even on ties. RANK = ties share, then skips (1,1,3). DENSE\_RANK = ties share, no gap (1,1,2).

### Nth-highest salary (the classic)

```
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rk FROM employees
) x WHERE rk = 2;
-- 2nd highest = 71000
```

### Running total & LAG/LEAD

```
SELECT trade_date, qty*price AS value,
       SUM(qty*price) OVER (ORDER BY trade_date, trade_id) AS running_total
FROM trades;

SELECT symbol, trade_date, price,
       LAG(price) OVER (PARTITION BY symbol ORDER BY trade_date) AS prev_price,
       LEAD(price) OVER (PARTITION BY symbol ORDER BY trade_date) AS next_price
FROM trades;
```

## 17. CTEs (Common Table Expressions)

```
-- Readability: employees above their OWN dept average
WITH dept_avg AS (
  SELECT dept, AVG(salary) AS avg_sal FROM employees GROUP BY dept
)
SELECT e.name, e.dept, e.salary
FROM employees e JOIN dept_avg d ON e.dept = d.dept
WHERE e.salary > d.avg_sal;
-- Asha, Carol, Ella

-- Recursive: org hierarchy levels
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 1 AS lvl FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.lvl+1
  FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT name, lvl FROM org ORDER BY lvl;
```

**CTE vs subquery:** A CTE is a named, reusable subquery declared with WITH — more readable, can be referenced multiple times, and supports recursion. Functionally similar to a derived table.

## 18. Built-in Functions Cheat Sheet

### String

```
UPPER(s) LOWER(s)
LENGTH(s)
SUBSTRING(s,1,3)
CONCAT(a,b) -- or a||b
TRIM(s) LTRIM RTRIM
REPLACE(s,'a','b')
LEFT(s,3) RIGHT(s,2)
```

### Date

```
CURRENT_DATE NOW()
EXTRACT(YEAR FROM d)
DATEDIFF(d1,d2)
DATE_ADD(d, INTERVAL 1 DAY)
DATE_TRUNC('month', d)
```

## Numeric

```
ROUND(x,2) CEIL(x) FLOOR(x)
ABS(x) MOD(a,b)
POWER(x,2) SQRT(x)
```

## Conditional / NULL

```
CASE WHEN ... THEN ... ELSE ... END
COALESCE(a,b,0) -- 1st non-null
NULLIF(a,b) -- NULL if a=b
ISNULL / IFNULL
```

```
-- CASE bucketing + pivot via conditional aggregation
SELECT client_id,
       SUM(CASE WHEN symbol='AAPL' THEN qty ELSE 0 END) AS aapl_qty,
       SUM(CASE WHEN symbol='MSFT' THEN qty ELSE 0 END) AS msft_qty
FROM trades GROUP BY client_id;
```

## 19. Views, Indexes, Procedures, Triggers

| Object                   | What it is                                         | BA-level why                                                                                    |
|--------------------------|----------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>View</b>              | Saved SELECT that behaves like a virtual table     | Simplify complex joins, restrict columns for security. <code>CREATE VIEW v AS SELECT ...</code> |
| <b>Materialized view</b> | View whose result is physically stored & refreshed | Faster reads on heavy aggregations (reporting).                                                 |
| <b>Index</b>             | Lookup structure on column(s)                      | Speeds reads (WHERE/JOIN/ORDER BY); slows writes; uses storage.                                 |
| <b>Stored procedure</b>  | Saved, reusable block of SQL logic                 | Encapsulate business logic, run as one call.                                                    |
| <b>Trigger</b>           | Code that auto-runs on INSERT/UPDATE/DELETE        | Audit trails, enforce rules (e.g., log every price change).                                     |

```
CREATE VIEW us_client_trades AS
  SELECT t.* FROM trades t JOIN clients c ON t.client_id=c.client_id
  WHERE c.country='US';

CREATE INDEX idx_trades_client ON trades(client_id); -- speed up joins/filters
-- Clustered index = table physically sorted by key (one per table).
-- Non-clustered = separate structure pointing to rows (many allowed).
```



## 20. Normalization

Organizing tables to reduce redundancy and avoid update/insert/delete anomalies.

| Form | Rule                                                                                              |
|------|---------------------------------------------------------------------------------------------------|
| 1NF  | Atomic values only — no repeating groups or multi-valued cells.                                   |
| 2NF  | 1NF + no partial dependency (non-key cols depend on the <i>whole</i> composite key).              |
| 3NF  | 2NF + no transitive dependency (non-key cols depend only on the key, not on another non-key col). |
| BCNF | Stronger 3NF — every determinant is a candidate key.                                              |

**Denormalization** deliberately reintroduces redundancy (merging tables, storing computed totals) to cut joins and speed up read-heavy reporting/data-warehouse queries. Trade-off: faster reads, harder writes & risk of inconsistency.

**Anomalies normalization prevents:** Update (change a value in many rows), Insert (can't add data without unrelated data), Delete (removing one fact loses another).

## 21. ACID & Transactions

| Property    | Meaning                                                     | Corporate-actions example                                        |
|-------------|-------------------------------------------------------------|------------------------------------------------------------------|
| Atomicity   | All steps commit or none do                                 | Dividend debit from issuer + credit to holders — all or nothing. |
| Consistency | DB moves from one valid state to another (constraints hold) | Total units after a stock split stay reconciled.                 |
| Isolation   | Concurrent transactions don't corrupt each other            | Two settlements on the same account don't interleave.            |
| Durability  | Committed data survives crashes                             | A booked trade persists after a system restart.                  |

**Isolation levels (low→high):** Read Uncommitted → Read Committed → Repeatable Read → Serializable. Higher = fewer anomalies (dirty/non-repeatable/phantom reads) but less concurrency.

**SQL vs NoSQL (one-liner):** SQL = structured, relational, schema-on-write, strong ACID (Oracle, SQL Server, Postgres). NoSQL = flexible schema, horizontal scale, eventual consistency (MongoDB, Cassandra).

## 22. Conceptual Q&A (rapid fire)

**Q1. DDL vs DML?** DDL defines/changes structure (CREATE/ALTER/DROP, auto-committed); DML changes data (INSERT/UPDATE/DELETE, needs COMMIT).

**Q2. WHERE vs HAVING?** WHERE filters rows before grouping (no aggregates); HAVING filters groups after aggregation.

**Q3. DELETE vs TRUNCATE vs DROP?** DELETE = rows, WHERE-able, rollback-able (DML). TRUNCATE = all rows, fast, resets identity, no rollback (DDL). DROP = removes the table itself.

**Q4. Primary vs Unique key?** PK: NOT NULL + unique, one per table. Unique: allows one NULL, many per table.

**Q5. Candidate vs Primary vs Alternate key?** Candidate = all minimal unique identifiers; Primary = the one chosen; Alternate = the remaining candidates.

**Q6. Primary vs Foreign key?** PK identifies a row within its table; FK references a PK in another table to enforce referential integrity.

**Q7. UNION vs UNION ALL?** UNION removes duplicates (sort cost); UNION ALL keeps them (faster).

**Q8. UNION vs JOIN?** UNION stacks rows vertically (same columns); JOIN combines columns horizontally by matching keys.

**Q9. INNER vs LEFT JOIN?** INNER returns only matches; LEFT returns all left rows plus matches (NULLs where none).

**Q10. Join vs Subquery?** Joins combine columns across tables (usually faster for returning data); subqueries suit existence checks / single values; correlated subqueries can be slow.

**Q11. Clustered vs Non-clustered index?** Clustered sorts the table physically (one per table); non-clustered is a separate lookup structure (many allowed).

**Q12. What is an index trade-off?** Faster reads, slower writes, extra storage.

**Q13. View vs Table?** A table stores data; a view is a saved query (virtual table) — no data of its own (unless materialized).

**Q14. SQL logical execution order?** FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT (why a SELECT alias can't be used in WHERE).

**Q15. Window function vs GROUP BY?** Both aggregate; GROUP BY collapses to one row per group, a window keeps every row and adds the value alongside.

**Q16. ROW\_NUMBER vs RANK vs DENSE\_RANK?** Unique numbering; ties-share-then-skip; ties-share-no-gap.

**Q17. Nth highest value?** DENSE\_RANK filtered to N, or MAX(x) WHERE x < (SELECT MAX(x)).

**Q18. CHAR vs VARCHAR?** CHAR = fixed length (space-padded); VARCHAR = variable length (stores only what's needed).

**Q19. Normalization vs Denormalization?** Normalize to remove redundancy (integrity, writes); denormalize to reduce joins (read/reporting speed).

**Q20. ACID?** Atomicity, Consistency, Isolation, Durability — the guarantees that keep transactions reliable.

**Q21. DELETE + IS NULL anti-join use?** LEFT JOIN + WHERE right.key IS NULL finds unmatched rows (e.g., unsettled trades, clients with no orders).

**Q22. Can you ROLLBACK a TRUNCATE?** Generally no — it's DDL and auto-commits (engine-dependent). DELETE can be rolled back within a transaction.

## 23. Practice Prompts (rehearse on the sample tables)

- Total notional per client, highest first → JOIN + SUM(qty\*price) + GROUP BY + ORDER BY.
- Each employee with their dept's average salary → window `AVG() OVER (PARTITION BY dept)`.
- Clients trading only AAPL → GROUP BY client HAVING `COUNT(DISTINCT symbol)=1 AND MAX(symbol)='AAPL'`.
- Largest trade per symbol → `ROW_NUMBER() OVER (PARTITION BY symbol ORDER BY qty*price DESC)=1`.
- Day-over-day change in daily notional → daily SUM in a CTE, then `LAG()`.
- Write the DDL for a `corporate_actions` table (PK, FK to a securities table, CHECK on ratio > 0, DEFAULT status 'ANNOUNCED').
- Second-highest-paid employee per department → DENSE\_RANK partitioned by dept, filter rk=2.
- Clients who traded AAPL but never MSFT → EXCEPT of two client\_id sets.